

Security & Compliance

Polycarpic — a virtual Single Family Office

Owner: `seceng` agent **Status:** Draft v1 — pending DevOps fold (backup key custody, §6.6 / §9) and Principal review

Traces to: PRD v1 (Approved 2026-09-27), ARCH v1, kickoff record **Last updated:** 2026-09-28

CONTENTS

1. Compliance Posture
2. Trust Boundaries
3. Threat Model (STRIDE)
4. Authentication & Authorization
5. Data Classification & Handling
6. Secret Management
7. Controls Catalog
8. CI/CD Security
9. Logging, Monitoring, Detection
10. Incident Response
11. Open Risks (Accepted)
12. Rulings on ARCH Open Questions

1. Compliance Posture

Source of truth is [PRD §6 — Non-Functional Requirements](#) → *Security / Regulatory*, decided with the Principal 2026-09-27.

The posture in one sentence: a self-hosted personal tool, operated by the person whose books it holds, has no *customer* and therefore no third-party compliance obligation — so no regime attaches, and the security work here is justified by the threat model, not by an auditor.

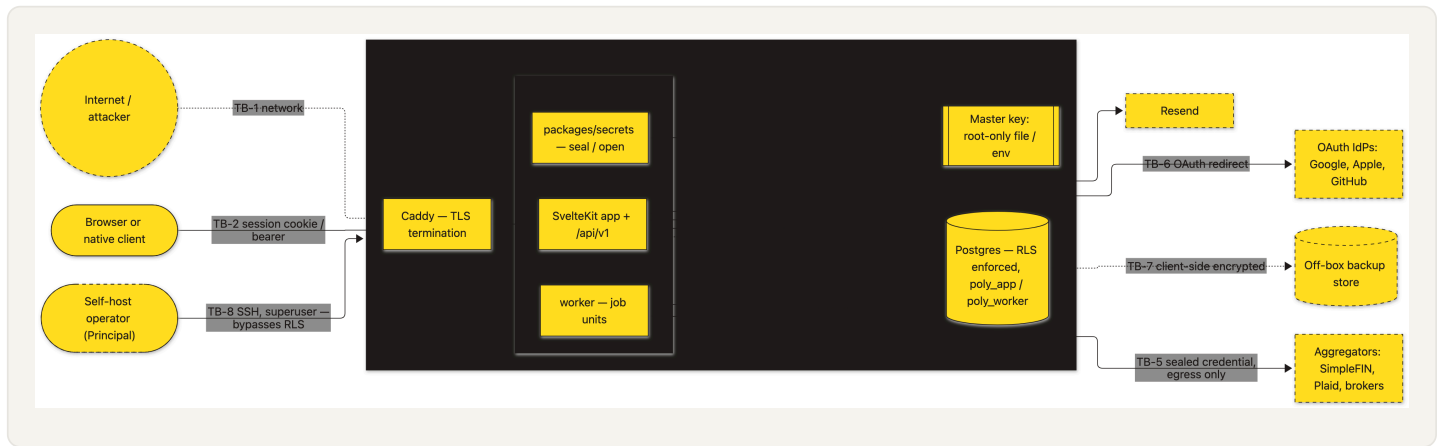
Regime	In scope?	Why	Target evidence
HIPAA	No	No health data is collected or processed. Medical <i>payees</i> may appear in transaction memos, which is why memo text is classified Confidential (§5) — but a payee string is not PHI held by a covered entity.	Data classification table (§5); memo fields in the log-redaction list (§9).
PCI-DSS	No	The system never touches a PAN, CVV or cardholder data. Aggregators (SimpleFIN, Plaid) mediate every bank and card connection; the system holds a read-only bearer credential, never card data.	<code>ImportAdapter</code> contract (ARCH §2.3): no card-data field exists in <code>RawTxn</code> .
GLBA (Safeguards Rule)	No for v1	GLBA attaches to a financial institution holding <i>customer</i> information. A self-hosted instance whose owner is also its only data subject has no customer relationship. Standing Non-Goal: the Principal will never host this for others (PRD §3 #10).	PRD §3 Non-Goal #10 and §6 audience decision. Re-opens only on a deliberate reversal (§11 R-6).
SOC 2	No	SOC 2 is a report sold to enterprise buyers. There are no buyers and no service organization.	n/a.
GDPR / CCPA	Adopted voluntarily	Not legally triggered (no controller/processor relationship with a third party), but the data-subject rights are cheap here and worth having. Scoped to the immutable-ledger model: <i>export</i> is complete; <i>delete</i> means account closure with actor anonymization, plus whole-entity purge by the sole owner — never a retroactive ledger rewrite.	PRD §5 : export requirement, account-deletion requirement, whole-entity purge requirement. Controls C-21 (provider-side revocation) and C-35–C-37 (purge guards) in §7.
State financial-privacy law (e.g. CA FIPA), PCI DSS direct	No — future trigger only	Would attach only if the standing Non-Goal is reversed and the Principal operates a hosted multi-user offering. Flagged so it is not a surprise later, not a v1 work item.	Accepted risk R-6 (§11).

What the absence of a regime does *not* buy us. The Principal has designated six items as hard requirements independent of any regime ([PRD §6](#)): credential encryption at rest with a provider-side revocation path; RLS on the membership join proven safe under pooling; a tested owner/manager/viewer permission matrix; DB-trigger-enforced

append-only posted rows; a first-class actor column; and documented disk encryption plus tested encrypted backups with a stated RPO/RTO. Each of those is a control in §7 with a named test, not a policy sentence.

2. Trust Boundaries

Derived from [ARCH §7](#) and the layer model ([ARCH §2.1](#)). Every boundary below is a STRIDE entry in §3 — that mapping is the doc's spine.



#	From	To	Boundary type	Control at the boundary
TB-1	Public internet	Caddy	Network	Firewall (22 restricted by source, 80/443 open, Postgres never published); automatic TLS with HSTS; Postgres and the app ports exist only on the compose network.
TB-2	Browser / native / future MCP client	SvelteKit app	Network + Authn	TLS 1.2+; BetterAuth session cookie (<code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Lax</code>) for web; revocable scoped bearer token for native and MCP clients; CSRF origin check on mutating routes.
TB-3	App / worker process	Postgres	Network + Authz	The main boundary. Connects as <code>poly_app</code> / <code>poly_worker</code> (<code>NOBYPASSRLS</code> , not the table owner); tenant identity set transaction-locally; every tenant table <code>FORCE s</code> RLS; default-deny. §4.3.
TB-4	<code>packages/secrets</code>	Master key (file / env)	Process + Filesystem	Key readable only by the app process user (mode <code>o600</code> , root-owned, outside the image and the repo); plaintext credentials exist only inside a <code>secrets.open()</code> call frame; never in Postgres, never in a log, never in a response. §6.
TB-5	Worker	Aggregators / brokers / price + BLS APIs	Network + Third-party trust	Egress only — no inbound webhook surface in v1; sealed credential opened per call; provider responses are untrusted input (schema-validated, stored as opaque payload, never rendered as HTML). §3.3.
TB-6	App	OAuth IdPs (Google / Apple)	Network + Delegated	OAuth 2.0 + PKCE via BetterAuth; per-operator client credentials held as operator

#	From	To	Boundary type	Control at the boundary
		/ GitHub), Resend	identity	secrets; email is a notification channel, never an authorization channel beyond BetterAuth's own verified flows.
TB-7	VPS	Off-box backup store	Network + Data egress	Backups encrypted <i>on the box before transmission</i> , with a key held independently of the box (§6.6, DevOps owns the mechanism in INFRA). The store credential is write-mostly.
TB-8	Operator (SSH)	VPS, Postgres superuser	Administrative	Key-only SSH, no password auth, source-restricted. Superuser bypasses RLS by definition — documented as accepted risk R-1 (§11), not a boundary RLS can defend.
TB-9	<code>poly_app</code> ordinary write path	Privileged purge path	Privilege escalation, intentional	One <code>SECURITY DEFINER</code> function owned by <code>poly_owner</code> , guarded by sole-owner check, fresh-export check, and an <code>audit_event</code> written before any delete. §4.5.
TB-10	Prod compose project	Staging compose project (same VPS)	Host-level (shared kernel)	Separate compose project, separate database, separate credentials; staging holds fake data only. Accepted risk R-4 (§11).

3. Threat Model (STRIDE)

One subsection per trust boundary from §2, plus two data-integrity surfaces that matter more here than any network threat. Not exhaustive — only threats with non-trivial likelihood × impact. Controls are catalogued in §7; the `C-n` references point there.

Adversary model. Three adversaries are in scope: (A) an unauthenticated internet attacker with no prior access; (B) an authenticated user of the instance who holds *some* membership and wants data or writes outside it — the household member with a viewer grant is the realistic case, and it is the adversary the whole tenancy design exists to stop; (C) a compromised or hostile upstream dependency (an aggregator returning malicious payloads, a poisoned npm package). Explicitly **out of scope**: the box's own root/superuser (R-1) and an attacker with physical access to a powered-on VPS.

3.1 TB-1 / TB-2 — Internet and client ↔ app

Threat	STRIDE	Likelihood	Impact	Control(s)
Credential stuffing / password spray against the login route	S	medium	high	C-1 (BetterAuth: hashed passwords, rate-limited login, MFA available), C-2 (OAuth preferred over password), C-20 (auth-outcome logging)
Session cookie stolen via XSS and replayed	S / E	low	high	C-3 (<code>HttpOnly</code> + <code>Secure</code> + <code>SameSite</code> cookie; Svelte escapes by default; CSP with no <code>unsafe-inline</code>), C-4 (session revocation on logout / password change / membership revocation)
Native/MCP bearer token leaked from a client device and replayed indefinitely	S	medium	high	C-5 (bearer tokens are named, listable and individually revocable by the user; stored hashed; scoped to the issuing user, never to an entity)
Cross-site request forgery on a mutating <code>/api/v1</code> route	T	low	medium	C-3 (<code>SameSite=Lax</code> + origin check on all non-GET routes; bearer routes are cookie-less by construction)
Unauthenticated request reads tenant data because a route forgot its auth check	I / E	medium	critical	C-6 (no route can leak: RLS default-deny means a query with no transaction context returns zero rows), C-7 (<code>withTenant()</code> is the only connection path; session-level <code>SET</code> banned by lint)
Request flood exhausts the single VPS	D	low	medium	Accepted: ~99% availability target (PRD §6). Caddy connection limits + fail2ban on SSH; no CDN/WAF in v1 (R-3).

3.2 TB-3 — App ↔ Postgres (the tenancy boundary)

This is adversary (B)'s whole playing field and the highest-value table in this document.

Threat	STRIDE	Likelihood	Impact	Control(s)
Pooled connection carries the previous request's	I	medium	critical	C-8 (<code>set_config('app.user_id', ..., is_local => true)</code> inside the transaction — the setting dies

Threat	STRIDE	Likelihood	Impact	Control(s)
identity; user B reads user A's books				at COMMIT/ROLLBACK; session <code>SET</code> forbidden), C-9 (pool-reuse test in the CI battery: A's transaction then B's on the same physical connection must show zero A rows)
A new table ships without an RLS policy — drafts, raw import rows, lots, <code>transfer_match</code> , staging are the easy misses	I	high	critical	C-10 (catalog-driven CI probe: any table with a tenant column and RLS not <code>ENABLE d + FORCE d</code> <i>fails the build</i> — coverage is derived, never a hand-maintained list), C-11 (inversion self-test: an unprotected canary table must leak, proving the probe has teeth)
App-layer <code>can()</code> check passes but the DB permits the write — authz enforced only in TypeScript	E	medium	high	C-12 (the database is the enforcement point ; <code>can()</code> is UX-level early rejection only), C-9 (per-role write probes in pgTAP: a viewer's INSERT must be rejected by policy, not by the app)
Viewer on entity X reads entity Y through a parent/child link ("the Household lens leaks")	I	medium	high	C-13 (policies consult <code>membership</code> only; <code>parent_id</code> is never in a <code>USING</code> clause — structure grants nothing), C-9 (a probe asserts a child-entity grant yields no parent rows and vice versa)
RLS helper functions are <code>SECURITY DEFINER</code> ; a mutable <code>search_path</code> lets a user shadow <code>membership</code> and widen visibility	E	low	critical	C-14 (every <code>SECURITY DEFINER</code> function pins <code>SET search_path = pg_catalog, app</code> and is <code>STABLE</code> ; <code>EXECUTE</code> granted narrowly; <code>CREATE</code> on <code>public</code> revoked from <code>poly_app</code>)
<code>poly_app</code> can read sealed credential ciphertext for entities it can see	I	medium	medium	Accepted by design — ciphertext without the master key is inert (§6). C-16 (column excluded from every DTO and OpenAPI response schema), C-17 (only <code>secrets.open()</code> may decrypt)
SQL injection via a payee string, report formula, or provider payload	T / I	low	high	C-18 (Drizzle parameterized queries only; raw SQL confined to <code>packages/db</code> migrations and reviewed), C-19 (report formula grammar is a parsed AST over an account allow-list, never interpolated SQL), plus C-6 — even a successful injection runs as <code>poly_app</code> under RLS with the caller's context

3.3 TB-5 — Worker ↔ aggregators and reference-data providers

Threat	STRIDE	Likelihood	Impact	Control(s)
Aggregator credential exfiltrated (log line, error message, API response, crash dump) and used to read the user's real bank data	I	medium	critical	C-15 (envelope encryption at rest), C-16 (never in a response DTO or the OpenAPI spec), C-20 (pino redaction list covers credential, token, access-URL and account-number keys; a redaction unit test asserts a seeded secret never appears in serialized output), C-17 (single decrypt path)
Unlinking an account deletes the local row but leaves the token live at the provider — standing read access to a bank the user believes is disconnected	I	high	high	C-21 (<code>ImportAdapter.revoked(cred)</code> is called <i>before</i> the local delete; failure surfaces an actionable error and leaves the connection in a <code>revocation_failed</code> state rather than silently vanishing; the attempt is an <code>audit_event</code>)
Hostile or compromised provider returns a payload that poisons the ledger (absurd amounts, script in a payee field, oversized JSON)	T	low	medium	C-22 (adapter output Zod-validated into <code>RawTxn</code> : <code>numeric</code> -parseable amount, bounded string lengths, payload size cap; rejected rows quarantine, never post), C-23 (nothing posts without user confirmation unless the user explicitly opted a rule into auto-post — draft-by-default is a security control, not only a UX one)
Provider payload rendered into the UI executes script (stored XSS via payee/memo)	T / E	low	high	C-3 (Svelte auto-escaping; <code>{@html}</code> banned by lint outside a reviewed allow-list; CSP)
Provider outage or rate-limit stalls imports; stale data is mistaken for current	D	medium	low	C-24 (last-synced timestamp surfaced per connection; backoff with retry; hash de-dup means a replayed window loses nothing)

3.4 TB-4 — The master-key boundary

Threat	STRIDE	Likelihood	Impact	Control(s)
Database dump or stolen backup yields plaintext aggregator credentials	I	medium	critical	C-15 (the master key never lives in Postgres — a DB-only compromise yields ciphertext; this is the single reason app-layer sealing beats pgcrypto here, §6.5)
Master key committed to the repo, baked into a container layer, or printed at startup	I	medium	critical	C-25 (secret scanning in CI and as a pre-commit hook), C-26 (key supplied only as a mounted file or env var; app logs the key <i>id</i> , never the key; startup fails closed if absent outside dev)

Threat	STRIDE	Likelihood	Impact	Control(s)
Key compromise has no recovery path because everything is encrypted under one static key forever	I	low	high	C-27 (per-record DEK wrapped by the master key + <code>key_id</code> column $\Rightarrow$ rotation is a rewrap job over small wrapped-DEK blobs, not a full re-encrypt; documented runbook, §6.4)
Dev/staging uses a weak or shared default key that reaches prod	S / I	medium	high	C-26 (no default key exists; dev uses an obviously-labelled ephemeral key generated per container and staging holds fake data only)

3.5 Ledger integrity — the repudiation and tampering surface

For a book of record, the interesting attacks are not reads. They are "this entry was never here", "this number changed", and "I did not do that".

Threat	STRIDE	Likelihood	Impact	Control(s)
A posted entry is edited or deleted — silently rewriting history	T / R	medium	critical	C-28 (<code>BEFORE UPDATE OR DELETE</code> trigger raises unconditionally on <code>journal_entry</code> , <code>journal_line</code> , <code>lot</code> , <code>lot_match</code> , <code>tax_period_freeze</code> and posted <code>raw_transaction</code> rows — DB-enforced, so even an ad-hoc <code>psql</code> session by the owner hits it ; corrections are reversal-only)
An unbalanced entry is written by a buggy or hostile code path	T	medium	high	C-29 (<code>DEFERRABLE INITIALLY DEFERRED</code> constraint trigger: $\Sigma \text{base_amount} = 0$ and ≥ 2 lines at COMMIT; property-based tests generate balanced/unbalanced entries)
A user denies having posted or confirmed an entry; or an auto-post rule's output is indistinguishable from a human's	R	medium	medium	C-30 (<code>actor_kind</code> $\in \{\text{user, rule, system}\}$ + <code>actor_id</code> , <code>NOT NULL</code> , defaulted from the transaction context so no code path can omit them — never inferred)
A line is posted to another entity's account, crossing the books	T / E	low	high	C-31 (composite FK (<code>entity_id</code> , <code>account_id</code>) forces every leg into the entry's entity; cross-entity transfers are two entries)
A raw import row is marked posted with no entry, or an entry points at nothing — breaking the audit chain	R	low	medium	C-32 (two-way link written in one transaction; check constraint: a posted raw row has an entry id and the entry's <code>source_ref</code> points back)

3.6 TB-9 — The privileged purge path

Threat	STRIDE	Likelihood	Impact	Control(s)
Purge is used as a general immutability bypass — an ordinary write path sets the purge GUC and edits history	T / E	low	critical	C-33 (the trigger honours <code>app.purge_entity</code> only when the current execution context is the <code>poly_owner</code> -owned definer function; <code>poly_app</code> setting the GUC directly achieves nothing), C-34 (a pgTAP test asserts exactly that: <code>poly_app</code> sets the GUC, the UPDATE still raises)
A co-owner or manager purges books out from under the others	T	low	critical	C-35 (sole-owner check inside the function: refuses if any other owner membership exists; manager and viewer cannot reach the path at all)
Purge destroys the evidence that the purge happened	R	medium	high	C-36 (<code>audit_event</code> is written <i>first</i> , sits outside the purge cascade, has a nullable <code>entity_id</code> , and is append-only — the record survives the thing it describes)
A user purges books they still owed themselves (or a tax authority) — irreversible data loss mistaken for a feature	—	medium	high	C-37 (mandatory completed export within the last 24 hours , verified by the function against an <code>audit_event</code> export record; re-authentication; explicit record-retention warning naming the user's own obligations)

3.7 TB-7 / TB-8 / TB-10 — Backups, operator access, co-located staging

Threat	STRIDE	Likelihood	Impact	Control(s)
Backup store credential leaks; an attacker downloads the full books	I	medium	critical	C-38 (client-side encryption <i>before</i> egress — the store never holds plaintext; the store credential is not a data-disclosure credential)
Backup encryption key is stored only on the box it backs up — the box dies and the backups are unreadable	—	high	critical	C-39 (key custody is off-box and independent of the VPS; the restore drill proves recoverability from a clean machine — DevOps owns the mechanism, §6.6)
Backups exist but have never been restored; RPO/RTO are aspirational	—	high	critical	C-40 (a tested restore is a release-gate item, not a doc sentence; stated RPO/RTO numbers land in INFRA — ARCH OQ-9)
Operator's SSH key is compromised — full superuser, RLS bypassed, master key readable	E	low	critical	Partly accepted (R-1). C-41 (key-only SSH, no password auth, source-restricted port 22, fail2ban), C-42 (IR runbook §10 treats this as SEV-1: rotate master key, rewrap, revoke every provider token)
Staging compromise pivots to prod on the shared kernel	E	low	high	Accepted (R-4). C-43 (separate compose project, separate DB credentials, fake data only)

Threat	STRIDE	Likelihood	Impact	Control(s)
				in staging — a staging compromise exposes no real books)

3.8 Supply chain and CI/CD

Threat	STRIDE	Likelihood	Impact	Control(s)
A malicious or compromised npm transitive dependency reads the master key or exfiltrates ledger rows at runtime	I / E	medium	critical	C-44 (lockfile committed, <code>--frozen-lockfile</code> installs, Dependabot/Renovate + <code>pnpm audit</code> gate), C-45 (SBOM generated per release so a named CVE can be answered in minutes), C-26 (key is env/file-scoped, not a build artifact)
A secret is committed to the repo (public GitHub)	I	medium	critical	C-25 (secret scan in CI <i>and</i> pre-commit; GitHub push protection), C-46 (no <code>.env</code> committed; <code>.env.example</code> carries names only)
CI deploy credentials (SSH key / registry token) leak from GitHub Actions	E	low	high	C-47 (deploy secrets are repo-environment-scoped with required reviewers on the prod environment; the deploy key is separate from the operator's own key and independently revocable)
An unreviewed change reaches <code>main</code> and auto-deploys to staging	T	low	medium	C-48 (branch protection: PR required, admins included, no force-push, <code>cairn</code> status check strict — already enforced, see CLAUDE.md)

4. Authentication & Authorization

Resolves ARCH OQ-2 (RLS shape + permission matrix) and OQ-3 (system-job principal). §4.3–§4.5 are the SecEng rulings the architect asked for; the summary table is §12.

4.1 Authentication

- **User authn.** BetterAuth against our own Postgres (no external identity store). Email+password with a modern memory-hard hash (BetterAuth default, Argon2id/scrypt class — *not* a fast hash), OAuth 2.0 + PKCE via Google / Apple / GitHub, TOTP MFA available with single-use recovery codes. Email verification and recovery flows go through Resend.
- **MFA policy.** Available and encouraged, **not** mandatory in v1 — a self-hoster locking themselves out of their own books with no support desk is a worse outcome than the marginal risk, given the box is single-tenant and the login surface is rate-limited. Recovery codes are shown once and stored hashed. Re-evaluate if a household ever exceeds ~5 members (R-7).
- **Session model (web).** Server-side session record in Postgres, referenced by an `HttpOnly / Secure / SameSite=Lax` cookie. Server-side means revocation is real: logout, password change, and MFA reset invalidate sessions immediately.
- **Token model (native / MCP).** Named, listable, individually revocable bearer tokens stored hashed, scoped to a *user* — never to an entity, because entity access must stay a live membership lookup. A token grants exactly what its user can do at the moment of the request, not at the moment of issue.
- **Service authn.** There is no service mesh: one Node process pair (web, worker) and one database. The worker authenticates to Postgres as `poly_worker`; on the Vercel target, Vercel Cron calls an internal route authenticated by a dedicated shared-secret header compared in constant time — that route is otherwise indistinguishable from an unauthenticated public endpoint and must be treated as a first-class auth surface.

4.2 Authorization model

RBAC over the membership join, enforced in the database. A user's rights come from rows in `membership` (`user_id`, `entity_id`, `role`) — never from a `user_id` column on a data table, and never from entity structure. *Access is a grant; the hierarchy grants nothing.*

Enforcement point: Postgres RLS. Single source of truth. The `can(role, action)` helper in `packages/tenancy` exists to render a correct UI and to return a clean 403 early. It is **not** an enforcement point, and no control in §7 depends on it. The design test is blunt: if every `can()` call were deleted, the system would still be safe — slower and uglier, but safe. Any control that fails that test is app-layer-only authz and gets rejected at the security gate.

4.2.1 Permission matrix (owner / manager / viewer)

Roles are uniform across entity types — a Trust's manager and a Household's manager hold the same rights. Entity *type* affects tax routing and reporting, never authorization; adding a type must never require touching a policy. **Every row below is a pgTAP assertion in the CI battery** (C-9): allowed actions succeed, denied actions are rejected *by policy*, with the app layer bypassed.

Action	Owner	Manager	Viewer	Enforced by
Read dashboard, balances, reports, posted entries	✓	✓	✓	RLS SELECT policy: <code>entity_id = any (app.visible_entities())</code>
Read full custodial account numbers	✓	✓	✗ — masked to last-4	Column-level: a masking view / DTO projection chosen by role. Viewers get enough to recognize an account, not enough to transact against it.
Read raw provider payloads (<code>raw_transaction.payload</code>)	✓	✓	✗	RLS policy requires <code>app.has_role(entity_id, 'manager')</code> ; the payload is the least-curated, most PII-dense field in the system.
Create / edit / delete a draft	✓	✓	✗	RLS <code>INSERT/UPDATE/DELETE</code> policy on <code>draft</code> : manager+
Confirm a draft → post a journal entry	✓	✓	✗	RLS <code>INSERT</code> policy on <code>journal_entry</code> / <code>journal_line</code> : manager+
Reverse a posted entry	✓	✓	✗	Same as posting — a reversal is a new entry; nothing is edited.
Create / edit accounts (chart of accounts)	✓	✓	✗	RLS on <code>account</code> : manager+
Create / edit rules, including setting <code>auto_post</code>	✓	✓	✗	RLS on <code>rule</code> : manager+. Auto-post is a standing write grant — worth a second look if managers ever become less trusted than they are in a household.
Link a provider connection (store a credential)	✓	✓	✗	RLS on <code>provider_connection</code> : manager+. The linking user is recorded (§4.4).
Unlink a connection (provider-side revoke + delete)	✓	✓ — only connections they linked	✗	Policy + function check. A manager cannot revoke the owner's bank link; an owner can revoke anyone's.
Export the entity's complete books	✓	✓	✗	Privileged function, manager+; writes an <code>audit_event</code> . A full export is an exfiltration-shaped action and is logged like one.
Set entity metadata (name, tax classification, parent link)	✓	✗	✗	RLS on <code>entity</code> : owner only. Tax classification changes how filings route — it is an owner decision.
Grant / revoke / change a membership	✓	✗	✗	RLS on <code>membership</code> : owner only. Managers cannot grant access — otherwise manager silently equals owner.

Action	Owner	Manager	Viewer	Enforced by
Transfer or add an owner	✓	✗	✗	RLS on <code>membership</code> : owner only.
Purge the entity's books	✓ sole owner only	✗	✗	<code>SECURITY DEFINER</code> function with a sole-owner check + fresh export (§4.5).
Read the entity's <code>audit_event</code> rows	✓	✗	✗	RLS on <code>audit_event</code> : owner only. It is the record of who did what to the books.
Delete own user account	self	self	self	Always self-service, independent of role — but refused while it would orphan any entity where the user is sole owner (PRD §5).

Revocation semantics. Revoking a membership takes effect on the next transaction — there is no cached authorization anywhere, because visibility is recomputed per transaction from `membership`. Two consequences must be handled explicitly: (a) the revoked user's active sessions and bearer tokens remain valid for *their own* entities and simply stop seeing the revoked one (correct, and free); (b) a native client's **local cache** is outside our control — the iOS/macOS milestone must purge cached entity data on a 403/empty-scope response, and SECURITY reviews that milestone specifically. Server-side we cannot un-see what was already read; the honest statement to the user is that revocation stops future access, not past knowledge.

4.3 RLS mechanism — SecEng ruling on ARCH §3.3 (OQ-2)

Ruling: ARCH §3.3 is confirmed as specified, with four additions (a)–(d) below. Transaction-local GUC, `STABLE SECURITY DEFINER` visibility functions, `FORCE` RLS everywhere, default-deny. The pgbouncer/pooling risk I flagged in Research is answered by `set_config(..., is_local => true)`: the setting is scoped to the transaction and discarded at COMMIT or ROLLBACK, so it is safe under both the in-process `node-postgres` pool and a transaction-mode external pooler.

Non-negotiable corollary: the whole guarantee rests on *transaction-mode* pooling. If an external pooler is ever introduced in *session* mode, or if any code opens a connection outside `withTenant()` / `withSystem()`, the guarantee is void. Hence: `SET` (session-level) is banned by lint; direct `pool.query()` outside `packages/db` is banned by lint; and a pooler-mode assertion belongs in the deploy checklist (DevOps, INFRA).

Additions to the ARCH shape:

- (a) Coverage must be derived, not listed.** The CI probe enumerates tables from `pg_catalog` and fails on any table carrying a tenant column without `ENABLE + FORCE` RLS and at least one policy. A hand-maintained list of "tables to check" will drift the first time someone adds a staging table in a hurry. Tables intentionally global (`price_observation`, `cpi_observation`, `tax_table`, `system_report_definition`) sit on an explicit, commented allow-list — and adding to that list is a security-gate decision, not a build fix.
- (b) `poly_app` must not own tables and must not hold `CREATE on public`.** ARCH says the first part; I am adding the second, because `CREATE` on a schema in the `search_path` is the standard way to defeat a `SECURITY DEFINER` function. Every definer function also pins its own `search_path` (C-14).
- (c) Write policies must be split per command.** A single `FOR ALL` policy with a `USING` clause and no `WITH CHECK` lets a manager *insert* a row attributed to an entity they cannot see. Every tenant table gets separate `SELECT` /

`INSERT` / `UPDATE` / `DELETE` policies, and every write policy carries an explicit `WITH CHECK`. A pgTAP test asserts the `WITH CHECK` path directly (insert with a forged `entity_id` must fail).

4. **(d) The probe needs a negative control.** Keep ARCH's inversion self-test (an unprotected canary table must leak). A green RLS suite that would also be green against a database with RLS switched off is worthless, and that failure mode is common enough to be worth one canary table.

Default-deny check. `current_setting('app.user_id', true)` returns NULL when unset, and `entity_id = any (app.visible_entities())` with a NULL-derived empty array matches nothing. The battery asserts a context-free query returns zero rows on every tenant table — this is what makes "a route forgot its auth check" a bug rather than a breach (C-6).

4.4 System jobs — SecEng ruling (OQ-3)

Ruling: adopt ARCH's proposal — a job's tenant writes run under the linking user's membership, with `actor_kind = 'system'` and `actor_id = <job>`. Reject a dedicated system policy clause.

Why. A "system may write anything" clause is a permanent RLS bypass sitting in every policy, reachable by any code path that can set the system flag, and it is exactly the kind of clause that gets copied into a new policy without thought. Binding the job to the linking user's membership means the job's write scope is bounded by construction — it can never write more than the human who created the connection could, and that bound is re-checked every transaction rather than at link time.

Consequences that must be built, not assumed:

- `provider_connection.linked_by_user_id` is `NOT NULL` and is the principal for that connection's jobs. `withSystem(job, connection, fn)` sets `app.user_id` to that user and `app.actor = 'system:<job>'`. **Actor and principal are deliberately different things** — the entries say "the import job did this", while the permission check says "under Alice's grant".
- **Fail closed on revocation.** If the linking user's membership is revoked or their account is deleted, the job's writes stop returning rows to write — correct, and automatic. But silent failure is unacceptable: the connection must flip to `principal_invalid`, surface on the entity's dashboard, and an `audit_event` is written. Re-linking by a current manager is the remedy. **This is a new requirement on the import milestone** and needs a QA test.
- Reference-data jobs (prices, CPI, tax tables) touch no tenant table and run as `poly_worker` with no user context — those policies grant `poly_worker` directly. That is the *only* place a role-based grant substitutes for a membership check, and it is safe precisely because those tables have no tenant column.
- A job must never run with an *absent* context as a fallback. If no principal resolves, the job errors — it does not proceed context-free (which, given default-deny, would silently write nothing and look like success).

4.5 The purge path — SecEng ruling (OQ-4)

Ruling: adopt ARCH §3.4's `SECURITY DEFINER` function. Reject a separate purge role; reject tombstoning.

- **Why not a separate role.** A purge role needs a credential; a credential needs to live somewhere the app can reach; anything the app can reach is reachable by an attacker who has the app. It converts a narrow code path into a standing credential — strictly worse.

- **Why not tombstoning.** Tombstoning is soft-delete with better branding. PRD §5 promises the owner that the books are *gone*, after an explicit warning about their own retention obligations; leaving the rows in place and hiding them behind a flag makes that promise false, and adds a permanent "show tombstoned rows" bypass to every query. Deletion must actually delete.
- **Why the definer function is acceptable.** It is one auditable code path, owned by `poly_owner`, with its escalation bounded to a single `entity_id` for the duration of one transaction. The append-only triggers honour `app.purge_entity` *only* inside that definer context — so `poly_app` setting the same GUC is inert, and C-34 tests exactly that.

Required guards, in order, inside the function:

1. Re-authentication within the last 5 minutes (checked at the app layer and recorded; the function receives the assertion and requires it present).
2. **Sole-owner check** — refuse if any other `owner` membership exists on the entity.
3. **Fresh-export check** — an `audit_event` of kind `export.completed` for this entity, by this user, within the last 24 hours. Stale exports do not count; an export that failed mid-stream does not count. The window exists so the exported file actually reflects what is about to be destroyed.
4. Write the `audit_event` of kind `entity.purged` — actor, timestamp, entity name and id, the export event it relies on, and row counts per table. **Written before the first delete**, outside the purge cascade, with a nullable `entity_id` so the FK does not drag it down.
5. Set `app.purge_entity` transaction-locally, delete in dependency order, reset.
6. Revoke every provider credential for that entity at the provider before deleting the connection rows (a purge that leaves live bank tokens behind is the worst possible outcome).

Not permitted: purge by a background job, purge by API without an interactive confirmation, purge of a partial date range (that is an edit, and edits are reversals), and purge while any other owner exists.

5. Data Classification & Handling

Three sensitive tiers plus one public tier. The tier drives the handling rule; the handling rule is testable. Tier assignment for a new field is a security-gate question, not an author's judgment call.

Tier	Data	Classification	At rest	In transit	Retention
T1	Authentication secrets — password hashes, OAuth refresh tokens, TOTP seeds, MFA recovery codes, session records, native/MCP bearer tokens, the Vercel Cron shared secret.	Restricted	Passwords and bearer tokens: one-way hashed (never recoverable). TOTP seeds and OAuth refresh tokens: encrypted via BetterAuth's storage on an encrypted volume. Never in application logs.	TLS 1.2+ only. Never in a URL, query string, or redirect fragment we control.	Sessions: BetterAuth TTL, revoked immediately on logout / password change / MFA reset. Recovery codes: until used or regenerated. Password hash: life of the account.
T2	Aggregator credentials — Plaid access tokens, SimpleFIN access URLs, broker OAuth tokens. <i>Bearer-equivalent to read access on the user's real bank accounts.</i>	Restricted — highest-value data in the system	Envelope-encrypted by packages/secrets (\$6): AES-256-GCM under a per-record DEK, DEK wrapped by the master key, stored as ciphertext + nonce + wrapped DEK + <code>key_id</code> . The master key is never in Postgres.	TLS to the provider only. Never sent to a browser or native client, in any form, ever — not masked, not partial.	Until unlink. Unlink revokes at the provider <i>first</i> , then deletes the row (C-21). Purge revokes before deleting (\$4.5).
T3	Financial data and PII — balances, transactions, holdings, lots, journal lines, custodial account numbers, and <i>payee / memo free text</i> . Memos routinely carry merchant names, medical payees, employer and income details.	Confidential	Postgres on an encrypted volume (DevOps/INFRA). No field-level encryption — it would break the ledger's query model (sums, ranges, reports) for a threat already covered by volume encryption plus RLS. Custodial account numbers are masked to last-4 for viewers (\$4.2.1).	TLS. Excluded from logs by the pino redaction list — memo and payee are treated as secrets for logging purposes even though they are not secrets for storage purposes (C-20).	Indefinite — the ledger is the record. Removal only via user-account deletion (actor anonymized, entries retained) or sole-owner whole-entity purge (\$4.5).

Tier	Data	Classification	At rest	In transit	Retention
T4	Reference data — EOD prices, CPI-U series, per-jurisdiction tax tables, system report templates.	Public / Internal	Plain. No tenant column by design; global to the instance.	TLS (inbound fetch).	Indefinite; historical series are append-mostly.

The inference rule that makes T3 real: which securities someone holds, where they shop, who they pay, and what they earn is a complete personal profile. The system therefore holds no analytics, no telemetry containing tenant data, and makes no outbound call carrying user data except a provider call the user initiated or an owner-initiated export. *Nothing about a user's books leaves the box otherwise* — that sentence from ARCH §1 is a control, and any future feature that breaks it (crash reporting with payloads, hosted error tracking, an LLM call carrying transaction text) requires an explicit security review and a PRD-level decision, not a library import.

A note on the log/store asymmetry. T3 is stored in the clear and redacted in logs. That is deliberate: the database is access-controlled, backed up encrypted, and RLS-filtered; log files are none of those things — they get tailed over SSH, pasted into bug reports, and shipped to whatever aggregator someone adds later. Logs are the most likely place for financial PII to escape, which is why the redaction list carries a unit test (C-20) rather than a convention.

6. Secret Management

Resolves *ARCH OQ-1*. Two distinct populations: **operator secrets** (one set per instance, held by the person running the box) and **user credentials** (per connection, held in the database, sealed).

6.1 Inventory

Secret	Population	Lives in	Reachable by	Rotation
Credential master key (AES-256)	Operator	Root-owned file (o600) mounted into the app container, or an env var. Never in the image, repo, or database.	The app process only, via <code>packages/secrets</code>	Operator-initiated rewrap; annually or on suspected compromise (§6.4)
Postgres role passwords (<code>poly_app</code> , <code>poly_worker</code> , <code>poly_migrator</code>)	Operator	Root-only env file on the VPS; Vercel/Neon env vars on the secondary target	Respective container only	On operator change or compromise
Provider client secrets (Plaid client id/secret, broker OAuth clients), price-provider API key, BLS key, Resend key, OAuth IdP client secrets	Operator	Same env file	App / worker	Per provider policy; on compromise
Backup-store credential + backup encryption key	Operator	Backup key held off-box (§6.6); store credential in the env file	Backup job	DevOps sets cadence in INFRA
Vercel Cron shared secret (secondary target only)	Operator	Vercel env var	The internal cron route	On compromise
CI deploy key / registry token	Operator	GitHub environment secrets, prod environment gated	Actions runner	On compromise; independently revocable from the operator's own SSH key
Aggregator access tokens / URLs	User (per connection)	Postgres, sealed (§6.3)	<code>secrets.open()</code> only	Provider-driven; rewrapped on master-key rotation

6.2 Operator secret handling

- One root-only env file on the VPS (0600 , outside the repo and outside any build context), referenced by compose. On the Vercel target, Vercel environment variables.
- **No secret is ever baked into a container image** — an image layer is effectively public once pushed to any registry.
- `.env.example` carries variable *names* and a comment each; never a value, never a placeholder that looks like a value.
- The app **fails to start** when a required secret is missing outside development. A system that silently degrades to "no encryption" because a variable was unset is worse than one that refuses to boot (C-26).
- Secrets are read once at startup into process memory; there is no runtime secret-fetch path and no admin endpoint that reveals configuration.

6.3 Credential sealing — the ruling (OQ-1)

Ruling: application-layer envelope encryption in `packages/secrets` , AES-256-GCM, per-record DEK wrapped by an operator-held master key. Reject pgcrypto; reject an external secrets manager for v1.

Scheme. On `seal(plaintext)` : generate a random 256-bit DEK and a random 96-bit nonce; encrypt the credential with AES-256-GCM under the DEK, binding the record id and provider as additional authenticated data (so a ciphertext cannot be moved between connection rows); wrap the DEK with the current master key (also AES-256-GCM, its own nonce); store `{ciphertext, nonce, wrapped_dek, dek_nonce, key_id, alg}` . On `open(sealed)` : unwrap with the master key version named by `key_id` , decrypt, return the plaintext *inside the call frame only*. AEAD is required, not optional — it makes ciphertext substitution a decryption failure rather than a silent success.

Property	Rule
Plaintext lifetime	Exists only within a <code>secrets.open()</code> caller frame, for the duration of one provider call. Never assigned to a module-level variable, never cached, never returned up more than one layer.
Logging	The sealed column and every known credential key name are in the pino redaction list. A unit test seals a known canary string and asserts it never appears in serialized log output (C-20).
API exposure	The column is absent from every DTO and from <code>openapi.json</code> . An OpenAPI-freshness diff in CI means adding it would show up in review as a visible spec change (C-16).
Errors	Provider errors are re-thrown with the credential stripped; a raw provider exception is never propagated to a response or a log line without passing through the scrubber.
Who can decrypt	Only <code>packages/secrets</code> . A lint boundary rule forbids any other package importing the low-level crypto module.

6.4 Key rotation for an operator with no KMS

Envelope encryption exists here for exactly one reason: **rotation without re-encrypting the data**. Rotating the master key rewraps a few dozen small DEK blobs rather than decrypting and re-encrypting every credential, which keeps the operation short, interruptible, and safe to retry.

1. Operator generates a new master key and adds it as version $n+1$, keeping n present. Both are loaded; $n+1$ becomes the write key.

2. `pnpm secrets:rewrap` — for each sealed record with `key_id = n` : unwrap the DEK with n , rewrap with $n+1$, update `key_id` . Idempotent and resumable; the credential ciphertext itself is untouched.
3. The job asserts zero rows remain at `key_id = n` , then the operator removes key n . An `audit_event` records the rotation.
4. **Cadence:** annually as hygiene; **immediately** on any suspicion that the box, the operator's SSH key, or a backup was compromised. In that case rotation alone is insufficient — see §10: assume the old key is known and revoke every provider credential.

Lost master key = unrecoverable credentials, by design. The remedy is re-linking each provider connection, which is inconvenient but not data loss — the ledger itself is not encrypted under this key. The runbook must say so plainly so a panicked operator does not go looking for a backdoor that must not exist.

6.5 Why not pgcrypto, and why not a secrets manager

- **pgcrypto (rejected).** To encrypt or decrypt in SQL, the key must be reachable by the Postgres process — passed as a query parameter (landing in `pg_stat_activity` and potentially the statement log) or held in a GUC/config file next to the data. That collapses the distinction between "database compromised" and "bank credentials compromised", which is the *entire* value of sealing. A stolen dump or a leaked backup must yield inert ciphertext; with pgcrypto and a co-located key, it does not.
- **External secrets manager — Vault, AWS/GCP KMS (rejected for v1).** The deploy audience is a competent stranger on one VPS (PRD §6). Requiring them to run or subscribe to a KMS contradicts the self-host goal and the near-zero-spend guardrail (PRD §2 goal 4), and it adds an availability dependency: a KMS outage would stop imports. The seam is preserved — `seal()` / `open()` with a `key_id` is exactly the interface a KMS-backed implementation would present, so swapping the implementation later touches one package. Revisit if the hosted-offering Non-Goal is ever reversed (R-6).
- **The credential-free path stays the recommended default.** CSV/OFX file import implements the same `ImportAdapter` interface with *no stored credential at all* (ARCH §2.3). An instance that never links an aggregator holds zero T2 data and removes this entire threat class. That should be the documented default for a cautious self-hoster, with aggregator linking presented as a convenience trade-off — stated in the operator runbook, not buried here.

6.6 Backup key custody — open, DevOps owns

Backups must be encrypted **on the box before egress** (C-38), so the off-box store never holds plaintext. The key custody question is the hard half and it is genuinely open: a key stored only on the VPS is useless in the exact scenario backups exist for. SecEng's requirement on DevOps (ARCH OQ-9, INFRA v1): **the encryption key must be recoverable independently of the VPS** — an asymmetric scheme where the box holds only a public key and the private key lives with the operator (age/gpg-style) satisfies this cleanly and is the recommended shape, since a write-only backup credential plus a public encryption key means a fully compromised box cannot read its own historical backups. A tested restore from a clean machine (C-40) is the acceptance criterion; RPO/RTO numbers are DevOps's to state.

7. Controls Catalog

Every control answers a specific threat in §3. Status is planned for all of them — no application code exists yet; this catalog is the checklist the Validate gate runs against, milestone by milestone. "Lives at" names the intended home from [ARCH §2.2](#).

#	Control	Threat	Lives at	Tested by	Milestone
C-1	Memory-hard password hashing; rate-limited login (per-account and per-IP); MFA (TOTP) available with hashed single-use recovery codes.	3.1 spray	<code>packages/auth</code>	Integration: N failed logins → lockout/backoff	M1
C-2	OAuth 2.0 + PKCE for Google / Apple / GitHub; no implicit flow; state parameter verified.	3.1	<code>packages/auth</code>	Playwright smoke (one provider)	M1
C-3	<code>HttpOnly</code> + <code>Secure</code> + <code>SameSite=Lax</code> cookies; origin check on non-GET; CSP without <code>unsafe-inline</code> ; <code>{@html}</code> lint-banned outside a reviewed allow-list.	3.1, 3.3	<code>apps/web</code> hooks	Header assertion test; lint rule	M1
C-4	Server-side sessions, invalidated on logout, password change, and MFA reset.	3.1	<code>packages/auth</code>	Integration	M1
C-5	Bearer tokens: named, hashed at rest, listable and individually revocable, scoped to a user (never to an entity).	3.1	<code>packages/auth</code>	Integration: revoke → 401	M1 (issue), native milestone (use)
C-6	Default-deny RLS : a query with no transaction context returns zero rows on every tenant table.	3.1, 3.2	<code>packages/db</code> policies	pgTAP battery	M1
C-7	<code>withTenant()</code> / <code>withSystem()</code> are the only connection paths; session-level <code>SET</code> and direct pool access outside <code>packages/db</code> are lint errors.	3.2	<code>packages/db</code> + ESLint boundaries	Lint in CI	M1
C-8	Transaction-local identity via <code>set_config('app.user_id', ..., true)</code> ; discarded at COMMIT/ROLLBACK.	3.2 pooling	<code>packages/db</code>	pgTAP	M1
C-9	RLS battery : pool-reuse test, cross-tenant probes, per-role write probes covering every row of the §4.2.1 matrix, <code>WITH CHECK</code> forgery test.	3.2 (all)	<code>packages/db/tests</code> (pgTAP)	CI gate on any schema change	M1, extended per milestone
C-10	Catalog-driven coverage probe: any table with a tenant column lacking	3.2 missing policy	<code>packages/db/tests</code>	CI gate	M1

#	Control	Threat	Lives at	Tested by	Milestone
	ENABLE + FORCE RLS and a policy fails the build ; global tables on a commented allow-list.				
C-11	Inversion self-test: an unprotected canary table must leak, proving the probe has teeth.	3.2	packages/db/tests	CI gate	M1
C-12	Database is the authz enforcement point; can() is UX-only. Security review rejects any control whose safety depends on an app-layer check.	3.2 app-only authz	packages/tenancy (advisory) + policies (binding)	/security-review at each gate	ongoing
C-13	Policies consult membership only; parent_id never appears in a USING / WITH CHECK clause.	3.2 hierarchy leak	packages/db policies	pgTAP: parent/child isolation	M1
C-14	Every SECURITY DEFINER function is STABLE with a pinned search_path; CREATE on public revoked from poly_app; EXECUTE granted narrowly.	3.2 definer hijack	packages/db migrations	pgTAP catalog assertion over all definer functions	M1
C-15	Envelope encryption of aggregator credentials (AES-256-GCM, per-record DEK, wrapped by operator master key, AAD-bound to the record).	3.3, 3.4	packages/secrets	Unit: round-trip, tamper-detection, cross-record substitution rejected	M2 (first real adapter)
C-16	Sealed columns excluded from every DTO and from openapi.json; the OpenAPI-freshness diff makes any exposure visible in review.	3.3	apps/web routes	CI spec diff + a test asserting the field is absent from connection responses	M2
C-17	Single decrypt path: only packages/secrets may import the crypto module; plaintext never leaves the caller frame.	3.3, 3.4	ESLint boundaries	Lint in CI	M2
C-18	Parameterized queries only (Drizzle); raw SQL confined to reviewed migrations.	3.2 injection	all packages	Lint + review	M1

#	Control	Threat	Lives at	Tested by	Milestone
C-19	Report formula grammar is a parsed AST over an account allow-list — never string-interpolated SQL.	3.2 injection	<code>packages/reports</code>	Unit: malicious formula rejected	M5
C-20	Log redaction list covering credentials, tokens, access URLs, account numbers, payee and memo text; a canary test asserts a seeded secret never reaches serialized output.	3.3, §5	logger config (<code>apps/*</code>)	Unit (canary)	M1
C-21	Unlink calls <code>ImportAdapter.revoke()</code> <i>before</i> the local delete; failure leaves <code>revocation_failed</code> visible to the user; the attempt is an <code>audit_event</code> .	3.3 stale token	<code>packages/import</code>	Integration with the fake adapter (revoke fails → state + audit row)	M2
C-22	Adapter output Zod-validated into <code>RawTxn</code> : numeric-parseable amounts, bounded string lengths, payload size cap; invalid rows quarantine.	3.3 poisoning	<code>packages/import</code>	Unit with hostile fixtures	M2
C-23	Draft-by-default: nothing posts without explicit user confirmation unless a rule is explicitly opted into auto-post.	3.3	<code>packages/posting</code>	Integration	M2
C-24	Per-connection last-synced timestamp surfaced in the UI; retry with backoff; hash de-dup makes replay lossless.	3.3 stale data	<code>packages/import</code> , <code>apps/web</code>	Integration	M2
C-25	Secret scanning in CI <i>and</i> as a pre-commit hook; GitHub push protection on.	3.4, 3.8	<code>.github/workflows</code> , hooks	CI gate (seeded canary commit in a fixture)	M1
C-26	No default master key; app fails to start without one outside dev; the key id — never the key — is logged.	3.4	<code>packages/secrets</code>	Unit: startup refuses	M2
C-27	Per-record DEK + <code>key_id</code> ⇒ rewrap-based rotation; <code>secrets:rewrap</code> job, idempotent and resumable.	3.4	<code>packages/secrets</code>	Integration: rotate, assert zero rows at old key id, round-trip still works	M2
C-28	Append-only triggers on posted ledger rows — raised in the database, so ad-hoc <code>psql</code> hits them too.	3.5 tampering	<code>packages/db</code> migrations	pgTAP: UPDATE and DELETE both raise	M2

#	Control	Threat	Lives at	Tested by	Milestone
C-29	Deferred balance trigger: Σ <code>base_amount</code> = 0 and ≥ 2 lines at COMMIT.	3.5	<code>packages/db</code>	pgTAP + property-based tests	M2
C-30	<code>actor_kind</code> + <code>actor_id</code> NOT NULL on entries and drafts, defaulted from transaction context.	3.5 repudiation	<code>packages/db</code> , <code>packages/ledger</code>	pgTAP: insert without context fails	M2
C-31	Composite FK (<code>entity_id</code> , <code>account_id</code>) forces every leg into the entry's entity.	3.5	<code>packages/db</code>	pgTAP	M2
C-32	Two-way raw $\leftrightarrow$ entry link written in one transaction; check constraint enforces both directions.	3.5	<code>packages/db</code> , <code>packages/posting</code>	pgTAP + integration	M2
C-33	Purge GUC honoured only inside the <code>poly_owner</code> definer context.	3.6 bypass	<code>packages/db</code> triggers	pgTAP (see C-34)	GA
C-34	Negative test: <code>poly_app</code> sets <code>app.purge_entity</code> itself $\rightarrow$ UPDATE/DELETE still raises.	3.6	<code>packages/db/tests</code>	pgTAP	GA
C-35	Sole-owner check inside the purge function; manager/viewer cannot reach the path.	3.6	<code>packages/db</code> function	pgTAP: second owner $\rightarrow$ refused	GA
C-36	<code>audit_event</code> append-only, nullable <code>entity_id</code> , outside the purge cascade, written before the first delete.	3.6 repudiation	<code>packages/db</code>	pgTAP: purge then assert the event survives	GA
C-37	Purge requires re-auth + a completed export within 24 h + an explicit retention warning.	3.6 mistake	<code>packages/db</code> function, <code>apps/web</code>	Integration: stale export $\rightarrow$ refused	GA
C-38	Backups client-side encrypted before egress; the store never holds plaintext.	3.7	backup job (INFRA)	DevOps restore drill	M1 (infra)
C-39	Backup key custody independent of the VPS (asymmetric recommended, §6.6).	3.7	INFRA	Restore from a clean machine	M1 (infra)
C-40	Tested restore with stated RPO/RTO — a release-gate item, not a documented intention.	3.7	INFRA runbook	Drill, recorded	M1 (infra), repeated per release
C-41	Key-only SSH, no password auth, source-restricted port 22, fail2ban; Postgres never published.	3.7	INFRA	DevOps config review	M1 (infra)

#	Control	Threat	Lives at	Tested by	Milestone
C-42	IR runbook §10 with a named SEV-1 path for box/key compromise.	3.7	this doc + INFRA runbook	Tabletop at GA	GA
C-43	Staging isolated by compose project, database and credentials; fake data only.	3.7	INFRA	DevOps config review	M1 (infra)
C-44	Committed lockfile, <code>--frozen--lockfile</code> installs, Dependabot/Renovate, <code>pnpm audit</code> gate on high severity.	3.8	<code>.github/workflows</code>	CI gate	M1
C-45	SBOM (CycloneDX) generated per release build and stored as a release artifact.	3.8	CI	Artifact presence check	M1
C-46	No <code>.env</code> committed; <code>.env.example</code> holds names only.	3.8	repo	C-25 scan + review	done/ongoing
C-47	Deploy secrets scoped to a GitHub <i>environment</i> with required reviewers on prod; deploy key separate from the operator key.	3.8	GitHub settings	DevOps config review	M1 (infra)
C-48	Branch protection on <code>main</code> : PR required, admins included, no force-push, strict <code>cairn</code> check.	3.8	GitHub settings	Verified 2026-09-23 / 2026-09-25 (CLAUDE.md)	implemented
C-49	Container image scan (Trivy or equivalent) on the built <code>adapter-node</code> image; high-severity findings block deploy.	3.8	CI	CI gate	M1 (infra)

Gate rule. A feature touching auth, tenancy/RLS, credentials, provider integration, the purge path, or logging does not merge until `/security-review` has run on its diff and every control listed for that milestone is either green or explicitly deferred with an entry in §11. "Planned" is not a passing state at the milestone where the control is due.

8. CI/CD Security

Shape agreed with `devops-engineer` ; *INFRA* holds the runner-level detail. Maps to *ARCH §8's* "Security scan" stage, which this section expands.

- **Branch protection (implemented).** `main` requires a PR, includes admins, forbids force-push, and requires the strict `cairn` status check. No direct pushes — the workflow rule is enforced by GitHub, not by etiquette (C-48).
- **Signed commits / tags.** Commit signing is *not* required — the repo has one human author and per-agent worktree identities, so signature verification would prove little and break the agent workflow. **Release tags cut by `/merge-pr` should be signed**, since a tag is what triggers a prod deploy and is the one artifact worth binding to the operator. Recommended, not yet implemented — tracked as R-8.
- **Secret scanning.** Pre-commit hook plus a CI job (gitleaks-class), plus GitHub push protection. Pre-commit alone is insufficient (hooks are skippable); CI alone is too late (the secret is already pushed to a public repo) — both, deliberately (C-25).
- **Dependency scanning.** Committed lockfile, `--frozen-lockfile` installs, automated update PRs (Dependabot or Renovate), and a `pnpm audit` gate failing on high severity. Given adversary (C), the supply chain is the most probable route to the master key (C-44).
- **Image scanning.** The built `adapter-node` image is scanned before it is pushed or deployed; high-severity OS/library findings block the deploy (C-49).
- **SAST.** `tsc` strict, ESLint including the layer-boundary rules (which are themselves a security control — they are what keeps the crypto module and the connection path un-importable from elsewhere) and a security-rules plugin. No commercial SAST; it would not pay for itself at this size.
- **DAST / penetration testing.** None in v1 (R-3). The exposed surface is one Caddy-fronted app on one box, and the highest-value assertions — cross-tenant isolation, immutability, credential non-exposure — are covered far more precisely by the pgTAP battery than by a scanner.
- **SBOM.** CycloneDX generated at release build, stored with the release artifact, so "are we affected by CVE-X?" is answerable without reconstructing a historical dependency tree (C-45).
- **The DB battery is a security gate.** ARCH §8 lists it under correctness; it is also the primary tenancy control. When `packages/db` or any schema changes, the RLS + trigger battery must pass — a red battery blocks the merge regardless of what else is green.
- **Deploy credentials.** Scoped to a GitHub environment with required reviewers on prod; the deploy key is distinct from the operator's SSH key so it can be revoked without locking the operator out (C-47).

9. Logging, Monitoring & Detection

Detection capability here is deliberately modest — one operator, one box, no SIEM. What exists must therefore be high-signal and trustworthy.

- **What is logged.** Structured JSON (pino): request id, user id (or `system:<job>`), route, action, outcome, duration, and the entity id where relevant. Authentication outcomes (success, failure, MFA challenge, lockout) are logged at a dedicated level so they can be grepped without wading through request noise.
- **What is never logged.** T1 and T2 values, in any form. T3 free text — **payee and memo strings are redacted in logs** even though they are stored in the clear (§5). Full account numbers, provider payloads, request bodies on mutating routes. The redaction list is code with a canary unit test, not a convention (C-20).
- **Storage and retention.** Container stdout → host files, rotated (logrotate, size- and age-bounded). Local only; no log shipping in v1, which means no third party ever holds even redacted financial traffic. If log shipping is ever added, it is a §5-boundary change requiring security review.
- **The audit log is the real detection surface.** `audit_event` (append-only, owner-readable, outside the purge cascade) records: membership grant / revoke / role change, provider connection link / unlink / revocation attempt and outcome, full-entity export, entity purge, master-key rotation, and principal-invalidation of a connection. These are the actions that matter after the fact — the ones an attacker would need, and the ones a user would dispute. It lives in the database rather than in log files precisely because it must survive log rotation and be queryable years later.
- **Alerts.** v1 has uptime pinging only (DevOps). There is *no* automated alerting on authentication-failure spikes, authz denials, or anomalous egress — accepted as R-2, with the compensating control that the audit log makes after-the-fact reconstruction possible. The first alert worth adding, when one is added, is repeated auth failure followed by a success.
- **Postgres logging.** Slow-query log on. Statement logging stays **off** in prod: `log_statement = 'all'` would write payee and memo text — T3 data — into a plaintext file with different handling than the database it came from.

10. Incident Response

Written for the realistic case: one person, their own data, no pager. The value here is a pre-written decision order for a moment when nobody thinks clearly.

10.1 Roles and declaration

On call: the instance operator — for the primary instance, the Principal. There is no rotation and no escalation path; the runbook must therefore be executable by one person from a laptop. Any self-hoster inherits this role for their own instance, which is why the runbook ships with the app rather than living only here.

Severity	Criteria	First action
SEV-1	Suspected disclosure of T1 or T2 data (master key, box compromise, leaked backup, credential in a public log); <i>or</i> a cross-tenant read/write (RLS bypass); <i>or</i> a ledger-integrity violation (a posted row changed, an unbalanced entry landed).	Contain first, diagnose second. §10.2.
SEV-2	Availability loss (box down, database down, TLS expired); backups failing; imports failing silently for more than a day.	Restore service; then root cause. No credential rotation needed unless SEV-1 is also suspected.
SEV-3	A single provider connection broken, a stale price series, a failed job with a clean retry, a low-severity dependency CVE.	Normal backlog issue in cairn.

10.2 SEV-1 runbook — containment → eradication → recovery

- 1. Contain.** Take the app off the internet (stop the web container or drop 443 in the firewall) — the database keeps running so evidence and books survive. Do *not* rebuild or `docker compose down -v`: destroying state destroys the answer to "what happened".
- 2. Preserve.** Snapshot the volume and copy current logs off the box before any remediation. This is the step that is always skipped and always wanted later.
- 3. Assess exposure.** Which tier? T2 exposure (credential/master key/backup) is the gravest because it reaches the user's *actual bank accounts*, which are outside this system's control and cannot be restored by restoring a backup.
- 4. Eradicate — credentials.** On any suspicion that the box, an SSH key, or a backup was exposed: **revoke every aggregator credential at the provider** (Plaid `/item/remove`, SimpleFIN access revocation, broker token revoke) — do not merely delete local rows. Then rotate the master key (§6.4), every Postgres role password, every provider API key, the Resend key, OAuth client secrets, the backup-store credential, and the CI deploy key. Assume anything readable on the box is known to the attacker.
- 5. Eradicate — sessions.** Invalidate all sessions and bearer tokens; force re-authentication; reset MFA if T1 exposure is plausible.
- 6. Eradicate — access.** Rotate operator SSH keys; review `authorized_keys`, system users, cron, and the compose files for anything added.
- 7. Recover.** Rebuild the host from a known-good image and restore data from the last backup taken *before* the suspected compromise window; re-link provider connections by hand (the deliberate consequence of key rotation, §6.4). Verify integrity before reopening: RLS battery green, balance invariants green, `audit_event` reviewed for the window.

8. **Special case — RLS bypass or ledger-integrity violation.** Do not reopen on a patch alone. Write the failing pgTAP test first, watch it fail, fix, watch it pass, and add it permanently to the battery (C-9). An integrity bug that leaves no regression test behind will return.
9. **Notify.** No regulator requires notification (§1). Every person holding a membership on an affected entity is told what was exposed and what they should do (e.g. watch the linked accounts) — honestly, even when it is the operator's own fault, and especially when the operator is also the only user.

10.3 Post-incident

- Blameless postmortem within 5 business days: timeline, what the threat model got wrong, which control was missing or mis-sized.
- A decision entry in `process/DECISIONS.md` ; **every SEV-1 also amends this document** — a new §3 threat row and a new §7 control, or an explicit §11 acceptance. An incident that changes no artifact was not learned from.
- File the regression test as a cairn issue if it cannot ship with the fix.

11. Open Risks (Accepted)

Risks we live with — for now. Each has a rationale, a compensating control, a re-evaluation date, and an approver. Entries marked *pending* have not yet been put to the Principal and are SecEng's recommendation until they are.

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
R-1	The operator's superuser bypasses RLS entirely. Anyone with root on the VPS or a Postgres superuser session reads and writes every entity's books regardless of membership, and can read the master key file.	Structural, not a defect: the person running the box owns the hardware, the volume and the keys. Pretending RLS defends against the box's own administrator would be theater. PRD §6 states it explicitly.	Append-only triggers still fire for a superuser — history cannot be silently rewritten from <code>psql</code> (C-28). SSH hardened (C-41). <code>audit_event</code> records privileged actions taken through the app.	Only on a decision to host for others (see R-6)	Rich Mosko (PRD §6, 2026-09-27)
R-2	No automated security alerting. No detection of auth-failure spikes, authz-denial patterns, or anomalous egress; discovery of a live intrusion depends on the operator noticing.	One operator, one box, near-zero-spend guardrail (PRD §2 goal 4). A SIEM for a single-household instance costs more attention than it returns, and a noisy alert nobody reads is worse than none.	<code>audit_event</code> gives after-the-fact reconstruction of every consequential action (§9); auth outcomes logged at a dedicated level; uptime ping.	At GA, or on the first real incident	<i>pending</i> — SecEng recommendation
R-3	No DAST, WAF, or penetration test. Web-layer flaws are found by review and by the type system, not by an attacker simulator.	Small, single-purpose surface behind Caddy; no compliance regime requires it (§1); the highest-value properties (tenant isolation, immutability, credential non-exposure) are asserted far more precisely by pgTAP than by any scanner.	RLS battery as default-deny backstop (C-6); SAST + dependency + secret + image scanning (§8); this threat model reviewed each milestone.	Before any hosted offering; otherwise at GA	<i>pending</i> — SecEng recommendation
R-4	Staging co-located with prod on one VPS — shared kernel, so a container escape from staging reaches prod.	Saves a second box (ARCH §10). Staging holds only fake data, so a staging compromise discloses no real books; the value at risk is the pivot, not the data.	Separate compose project, database and credentials (C-43); staging never holds real provider credentials; ARCH	INFRA v1 (DevOps, ARCH OQ-9)	<i>pending</i> — inherited from ARCH §10

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
			already flags a later split.		
R-5	Master-key rotation is a manual operator runbook step , not automated or scheduled, and a lost key makes sealed credentials unrecoverable.	The target operator has no KMS (§6.5). Automation would require storing the new key somewhere the automation can reach — reintroducing the problem. Unrecoverability is the correct failure mode; the recovery is re-linking providers, not a backdoor.	Rewrap job is idempotent and resumable (C-27); the runbook states plainly that a lost key means re-linking; the ledger itself is not encrypted under this key.	Annually, with the rotation	<i>pending</i> — SecEng recommendation
R-6	Multi-tenant design leaves the hosted-offering door open . If the standing Non-Goal is ever reversed, GLBA Safeguards Rule, state financial-privacy law, and possibly PCI DSS attach — and R-1, R-2, R-3 all become unacceptable at once.	Hosting for others is a standing Non-Goal (PRD §3 #10), not a deferral. The architecture is tenant-safe by construction, so the reversal would be a compliance programme, not a rewrite.	This row exists so the reversal is a deliberate, costed decision. Any such decision re-opens §1 and invalidates R-1/R-2/R-3.	Only on a deliberate decision to host	Rich Mosko (PRD §3, 2026-09-27)
R-7	MFA is optional, not mandatory . A password-only account with a granted membership is a viable path to another person's financial data.	Self-hosted with no support desk: a forced-MFA lockout has no recovery path and destroys access to the user's own books. Login is rate-limited and OAuth (which carries the IdP's own MFA) is the encouraged path.	C-1 rate limiting + lockout; OAuth-first onboarding; hashed single-use recovery codes when MFA is enabled; auth outcomes logged.	When a household exceeds ~5 members, or at GA	<i>pending</i> — SecEng recommendation
R-8	Commits and release tags are unsigned . A repo-write compromise could introduce a commit indistinguishable	One human author plus per-agent worktree identities; signature verification would prove little about agent commits and complicates the workflow. Branch protection already prevents unreviewed merges (C-48).	Branch protection with admins included, no force-push, strict status check. Signing release tags is recommended before the first	Before the first prod deploy (M1 infra)	<i>pending</i> — SecEng recommendation

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	from an authored one.		prod deploy , since a tag triggers deployment.		
R-9	User-defined entity types are readable across tenants. A type created by one user is visible to anyone holding a membership on any entity of that type — a deliberate cross-tenant metadata read (kickoff §2.4).	Necessary: a viewer must read the type name to render the entity. The exposure is a type <i>label</i> and its owner id — no financial data, no membership list, no entity names.	An explicit, narrow RLS policy on <code>entity_type</code> rather than a blanket read grant; a pgTAP test asserts it exposes label-level fields only. Users are told a custom type name is visible to anyone sharing that type.	If entity types ever carry more than a label	<i>pending</i> — inherited from kickoff §2.4
R-10	Cross-root read access (PT-3, tooling): <code>cairn serve</code> reads directories named in a committed <code>roots:</code> config (or <code>--repos</code>), i.e. outside the repo. A malicious <code>roots:</code> entry in a PR could point a reviewer's local board at a sibling directory (e.g. <code>../../.ssh</code>).	Read-only and localhost-only — no write path to a secondary root, and the <code>/api/board</code> payload carries only parsed <code>cairn</code> frontmatter, so there is no exfiltration channel. A directory without a <code>config.yml</code> is skipped before anything is read. Whoever can commit a <code>roots:</code> entry already has a stronger vector (editing <code>cairn.py</code> / <code>board.js</code> directly), so a runtime path-jail would be theater against the real threat.	Startup stderr banner logs each resolved absolute root path when >1 root loads (human tripwire against a stale/fat-fingered <code>roots:</code> silently serving the wrong tracker). Re-evaluate if <code>cairn</code> ever gains a write path to non-primary roots.	2027-02-21	Rich Mosko

12. Rulings on ARCH Open Questions

The four questions [ARCH §12](#) assigns to [seceng](#), answered. Each links to the section carrying the reasoning; the architect can close OQ-1–OQ-4 against this table.

OQ	Question	Ruling	Detail
OQ-1	Credential sealing mechanism and key rotation for a self-hoster with no KMS.	App-layer envelope encryption in <code>packages/secrets</code> : AES-256-GCM, per-record DEK wrapped by an operator-held master key, AAD-bound to the record, stored as ciphertext + nonce + wrapped DEK + <code>key_id</code> . pgcrypto rejected (co-locates key with data, so a DB dump yields plaintext). External KMS rejected for v1 (contradicts self-host + near-zero-spend; adds an availability dependency) — the <code>seal / open / key_id</code> seam ARCH already fixed is exactly what a KMS implementation would slot into. Rotation is an operator-run rewrap job over wrapped DEKs, annually or on compromise.	§6.3, §6.4, §6.5; controls C-15, C-17, C-26, C-27
OQ-2	Confirm the RLS shape in ARCH §3.3, and the permission matrix per role × action × entity type.	ARCH §3.3 confirmed (transaction-local GUC, <code>STABLE SECURITY DEFINER</code> visibility functions, <code>FORCE RLS</code> , default-deny) with four additions: (a) coverage probe derived from <code>pg_catalog</code> , not a hand-maintained list; (b) <code>poly_app</code> also loses <code>CREATE</code> on <code>public</code> ; (c) per-command policies with an explicit <code>WITH CHECK</code> on every write — no <code>FOR ALL</code> ; (d) keep the inversion canary. Matrix: roles are uniform across entity types — type affects tax routing and reporting, never authorization. Managers post, confirm, reverse, link and export; only owners manage memberships, entity metadata and audit reads; viewers are read-only and see account numbers masked to last-4, with no access to raw provider payloads ; purge is sole-owner only.	§4.2.1 (matrix), §4.3 (mechanism); controls C-6 ... C-14
OQ-3	System-job principal: the linking user's membership, or a dedicated system policy clause?	Linking user's membership — ARCH's proposal adopted; a system policy clause is rejected as a permanent RLS bypass reachable by any path that can set the flag. <code>provider_connection.linked_by_user_id</code> is <code>NOT NULL</code> and is the principal; actor stays <code>system:<job></code> , so attribution and authority are deliberately separate. New requirement on the import milestone : when that user's membership is revoked or their account deleted, the connection flips to <code>principal_invalid</code> , surfaces on the dashboard, and writes an <code>audit_event</code> — it must not fail silently. Reference-data jobs (no tenant column) are the only role-granted exception.	§4.4; control C-30, audit entries §9
OQ-4	Purge path coexistence with the append-only triggers, and how the purge is audited.	ARCH §3.4's SECURITY DEFINER function adopted. Separate purge role rejected (converts a code path into a standing credential); tombstoning rejected (makes PRD §5's promise false and adds a permanent query bypass). Required guards, in order: re-auth ≤ 5 min → sole-owner check → completed export within 24 h → <code>audit_event</code> written <i>before</i> the first delete (append-only, nullable <code>entity_id</code> , outside the cascade) → transaction-local <code>app.purge_entity</code> honoured only inside the definer context → provider-side credential revocation before deleting connection rows . No purge	§4.5; controls C-33 ... C-37

OQ	Question	Ruling	Detail
		by job, by API without interactive confirmation, of a partial range, or while a second owner exists.	

12.1 Asks on other owners

- **architect:** ARCH §3.3 gains additions (a)–(d) from §4.3; ARCH §3.4 gains the guard order from §4.5; `provider_connection` gains `linked_by_user_id NOT NULL` and a `principal_invalid` state (§4.4); `raw_transaction.payload` is manager+ only, and `audit_event` is owner-read-only (§4.2.1). Trust boundary TB-10 (staging ↔ prod on one host) and TB-4 (master key ↔ process) are not in ARCH §7's list and should be.
- **devops-engineer:** backup key custody must be recoverable independently of the VPS — asymmetric (box holds the public key only) is the recommended shape (§6.6); a restore from a clean machine is the acceptance test (C-39, C-40). Also: confirm the pooler runs in *transaction* mode on both targets (§4.3), and scope prod deploy secrets to a reviewed GitHub environment (C-47).
- **qa-engineer:** every row of the §4.2.1 matrix is a pgTAP assertion; the battery needs the pool-reuse test, the `WITH CHECK` forgery test, the catalog-coverage probe, the inversion canary, and the purge negative test (C-34). The log-redaction canary (C-20) is a unit test that must exist before the first real adapter ships.
- **Principal (via team-lead):** R-2, R-3, R-5, R-7, R-8 and R-9 are accepted risks awaiting your approval; R-1 and R-6 you already approved in PRD §6/§3. The one with a real trade-off is **R-7 (MFA optional)** — the alternative is mandatory MFA with no lockout recovery.